

CSCI E-106: Assignment 7

Problem 1

```
data(fat, package="faraway")
str(fat)
```

```
## 'data.frame':    252 obs. of  18 variables:
## $ brozek : num  12.6 6.9 24.6 10.9 27.8 20.6 19 12.8 5.1 12 ...
## $ siri : num  12.3 6.1 25.3 10.4 28.7 20.9 19.2 12.4 4.1 11.7 ...
## $ density: num  1.07 1.09 1.04 1.08 1.03 ...
## $ age : int  23 22 22 26 24 24 26 25 25 23 ...
## $ weight : num  154 173 154 185 184 ...
## $ height : num  67.8 72.2 66.2 72.2 71.2 ...
## $ adipos : num  23.7 23.4 24.7 24.9 25.6 26.5 26.2 23.6 24.6 25.8 ...
## $ free : num  135 161 116 165 133 ...
## $ neck : num  36.2 38.5 34 37.4 34.4 39 36.4 37.8 38.1 42.1 ...
## $ chest : num  93.1 93.6 95.8 101.8 97.3 ...
## $ abdom : num  85.2 83 87.9 86.4 100 94.4 90.7 88.5 82.5 88.6 ...
## $ hip : num  94.5 98.7 99.2 101.2 101.9 ...
## $ thigh : num  59 58.7 59.6 60.1 63.2 66 58.4 60 62.9 63.1 ...
## $ knee : num  37.3 37.3 38.9 37.3 42.2 42 38.3 39.4 38.3 41.7 ...
## $ ankle : num  21.9 23.4 24 22.8 24 25.6 22.9 23.2 23.8 25 ...
## $ biceps : num  32 30.5 28.8 32.4 32.2 35.7 31.9 30.5 35.9 35.6 ...
## $ forearm: num  27.4 28.9 25.2 29.4 27.7 30.6 27.8 29 31.1 30 ...
## $ wrist : num  17.1 18.2 16.6 18.2 17.7 18.8 17.7 18.8 18.2 19.2 ...
```

Take the fat data, and use the percentage of body fat, siri, as the response and the other variables, except brozek and density as potential predictors.

```
cnames = names(fat)
brozek = match("brozek", cnames)
density = match("density", cnames)
fat_data = subset(fat, select = -c(brozek, density))
```

a-) Create train and test data sets. Use 70set and use remaining data (30(use set.seed(1023))).

```
set.seed(1023)
fat_pop_n = nrow(fat_data)
fat_alpha_n = round(7/10*fat_pop_n)
fat_training_idx = sample(fat_pop_n, size = fat_alpha_n, replace=FALSE)
fat_training = fat_data[fat_training_idx,]
fat_testing = fat_data[-fat_training_idx,]
fat_training_n = nrow(fat_training)
fat_testing_n = nrow(fat_testing)
print(sprintf("%i/%i=%f", fat_training_n, fat_pop_n, fat_training_n/fat_pop_n))
```

```
## [1] "176/252=0.698413"
```

Train each model and comment on the model performance.

a-) Linear regression with all predictors,

```
fat_lm = lm(siri ~ ., fat_training)
mse = mean(fat_lm$residuals^2)
print(mse)
```

```
## [1] 1.632889
```

```
summary(fat_lm)
```

```
##
## Call:
## lm(formula = siri ~ ., data = fat_training)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.7307 -0.5969  0.1482  0.8459  3.0884
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -65.17502   16.72815  -3.896 0.000143 ***
## age           0.01651    0.01291   1.279 0.202882
## weight       0.25847    0.04859   5.319 3.46e-07 ***
## height       0.86334    0.23072   3.742 0.000254 ***
## adipos       0.63010    0.33914   1.858 0.065010 .
## free        -0.59635    0.01564 -38.134 < 2e-16 ***
## neck         0.03618    0.08748   0.414 0.679698
## chest        0.10326    0.03959   2.608 0.009961 **
## abdom        0.07473    0.04067   1.838 0.067991 .
## hip         -0.03780    0.05482  -0.690 0.491434
## thigh        0.19678    0.05442   3.616 0.000400 ***
## knee         0.08862    0.10555   0.840 0.402403
## ankle        0.09482    0.07587   1.250 0.213228
## biceps       0.06670    0.06396   1.043 0.298532
## forearm      0.25327    0.07846   3.228 0.001512 **
## wrist        0.26320    0.22138   1.189 0.236242
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.34 on 160 degrees of freedom
## Multiple R-squared:  0.9745, Adjusted R-squared:  0.9722
## F-statistic: 408.3 on 15 and 160 DF,  p-value: < 2.2e-16
defaultSummary(data.frame(obs=fat_training$siri, pred=predict(fat_lm, fat_training)))

##      RMSE  Rsquared      MAE
## 1.2778453 0.9745390 0.9343454
```

The R-squared values are very high. The model may even be over-fitted.

There are positive and negative weights in the model, which can happen when there is multi-collinearity.

The MSE is 1.64.

b-) Linear regression with variables selected using stepwise (both ways) selection criteria,

```
fat_ols_step_both = olsrr::ols_step_both_adj_r2(fat_lm)
fat_ols_step_both
```

```

##
##
##                               Stepwise Summary
## -----
## Step      Variable      AIC      SBC      SBIC      R2      Adj. R2
## -----
## 0      Base Model      1235.795      1242.136      732.495      0.00000      0.00000
## 1      abdom (+)      1072.040      1081.551      567.138      0.61007      0.60783
## 2      free (+)      913.299      925.981      407.558      0.84356      0.84175
## 3      weight (+)      680.639      696.492      178.400      0.95876      0.95804
## 4      abdom (-)      679.645      692.327      178.019      0.95853      0.95805
## 5      forearm (+)      662.981      678.833      161.397      0.96270      0.96205
## 6      height (+)      645.471      664.494      144.341      0.96661      0.96583
## 7      chest (+)      637.593      659.787      136.749      0.96843      0.96751
## 8      thigh (+)      628.634      653.998      128.381      0.97034      0.96929
## 9      age (+)      619.154      647.688      119.814      0.97221      0.97106
## 10     adipos (+)      616.358      648.063      117.533      0.97296      0.97166
## 11     wrist (+)      614.403      649.279      116.116      0.97356      0.97213
## 12     ankle (+)      614.959      653.005      116.980      0.97378      0.97219
## -----
##
## Final Model Output
## -----
##
##                               Model Summary
## -----
## R      0.987      RMSE      1.297
## R-Squared      0.974      MSE      1.682
## Adj. R-Squared      0.972      Coef. Var      7.287
## Pred R-Squared      0.955      AIC      614.959
## MAE      0.939      SBC      653.005
## -----
## RMSE: Root Mean Square Error
## MSE: Mean Square Error
## MAE: Mean Absolute Error
## AIC: Akaike Information Criteria
## SBC: Schwarz Bayesian Criteria
##
##                               ANOVA
## -----
##                               Sum of
##                               Squares      DF      Mean Square      F      Sig.
## -----
## Regression      10991.416      10      1099.142      612.716      0.0000
## Residual      295.991      165      1.794
## Total      11287.406      175
## -----
##
##                               Parameter Estimates
## -----
## model      Beta      Std. Error      Std. Beta      t      Sig.      lower      upper
## -----
## (Intercept)      -66.507      16.128      -4.124      0.000      -98.351      -34.663
## free      -0.611      0.013      -1.461      0.000      -0.636      -0.585

```

```
##      weight      0.274      0.046      1.004      5.902      0.000      0.182      0.365
##    forearm      0.270      0.073      0.067      3.695      0.000      0.126      0.414
##      height      0.906      0.224      0.296      4.035      0.000      0.462      1.349
##       chest      0.126      0.036      0.128      3.482      0.001      0.055      0.198
##       thigh      0.219      0.048      0.146      4.532      0.000      0.124      0.315
##        age      0.026      0.012      0.040      2.195      0.030      0.003      0.050
##      adipos      0.706      0.331      0.313      2.134      0.034      0.053      1.359
##       wrist      0.346      0.211      0.040      1.642      0.103     -0.070      0.761
##       ankle      0.085      0.073      0.019      1.166      0.245     -0.059      0.230
## -----
```

```
defaultSummary(data.frame(obs=fat_training$siri, pred=predict(fat_ols_step_both$model, fat_training)))
```

```
##      RMSE Rsquared      MAE
## 1.2968289 0.9737769 0.9389624
```

The R-squared (0.974) and Adjusted R-squared (0.972) are almost identical to that of the full linear model. There are 10 variables in this model, as opposed to the 15 of the full model.

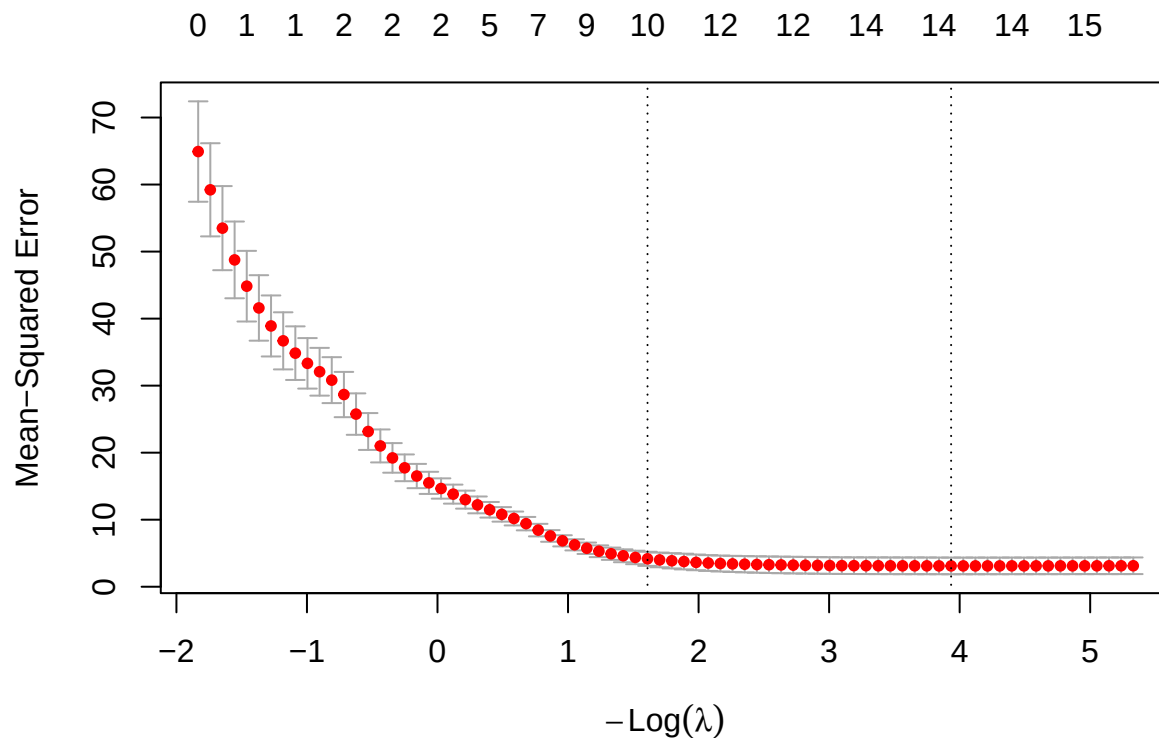
The MSE is 1.68.

c-) Ridge Regression

```
fat_training_response = fat_training$siri
fat_training_dm = model.matrix(siri ~ ., fat_training)[,-c(1)]
```

```
fat_ridge = cv.glmnet(
  fat_training_dm,
  fat_training$siri,
  nlambda=100,
  lambda.min.ratio=0.0001,
  type.measure="mse"
)
fat_ridge_lambda_min = fat_ridge$lambda.min
print(fat_ridge)
```

```
##
## Call: cv.glmnet(x = fat_training_dm, y = fat_training$siri, type.measure = "mse",      nlambda = 100)
##
## Measure: Mean-Squared Error
##
##      Lambda Index Measure      SE Nonzero
## min 0.01955    63   3.105 1.242         14
## 1se 0.20012    38   4.163 1.053         10
plot(fat_ridge)
```



```
defaultSummary(data.frame(
  obs=fat_training_response,
  pred=c(predict(fat_ridge, s=fat_ridge_lambda_min, fat_training_dm))
))
```

```
##      RMSE  Rsquared      MAE
## 1.2881482 0.9741931 0.9356602
```

The R-squared of 0.974 is almost identical to the linear models.

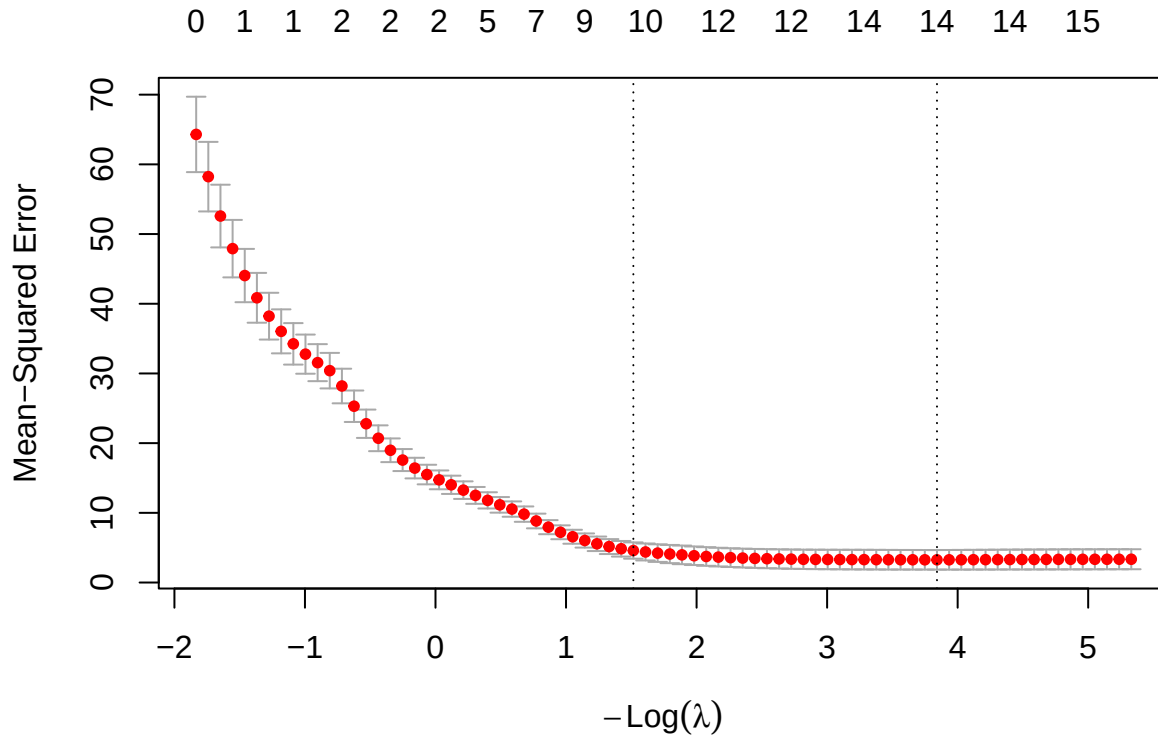
The MSE is 1.66.

d-) Perform Lasso Regression and comment on the model performance

```
fat_lasso = cv.glmnet(
  fat_training_dm,
  fat_training_response,
  alpha=1,
  nlambda=100,
  lambda.min.ratio=0.0001
)
fat_lasso_lambda_min = fat_lasso$lambda.min
print(fat_lasso)
```

```
##
## Call:  cv.glmnet(x = fat_training_dm, y = fat_training_response, alpha = 1,      nlambda = 100, lambda.min.ratio = 0.0001)
##
## Measure: Mean-Squared Error
##
##      Lambda Index Measure      SE Nonzero
## min 0.02146    62  3.257 1.402      14
## 1se 0.21963    37  4.592 1.169      10
```

```
plot(fat_lasso)
```



```
defaultSummary(data.frame(
  obs=fat_training_response,
  pred=c(predict(fat_lasso, s=fat_lasso_lambda_min, fat_training_dm))
))
```

```
##      RMSE Rsquared      MAE
## 1.2896325 0.9741469 0.9361102
```

The R-squared of 0.974 is almost identical to the linear models.

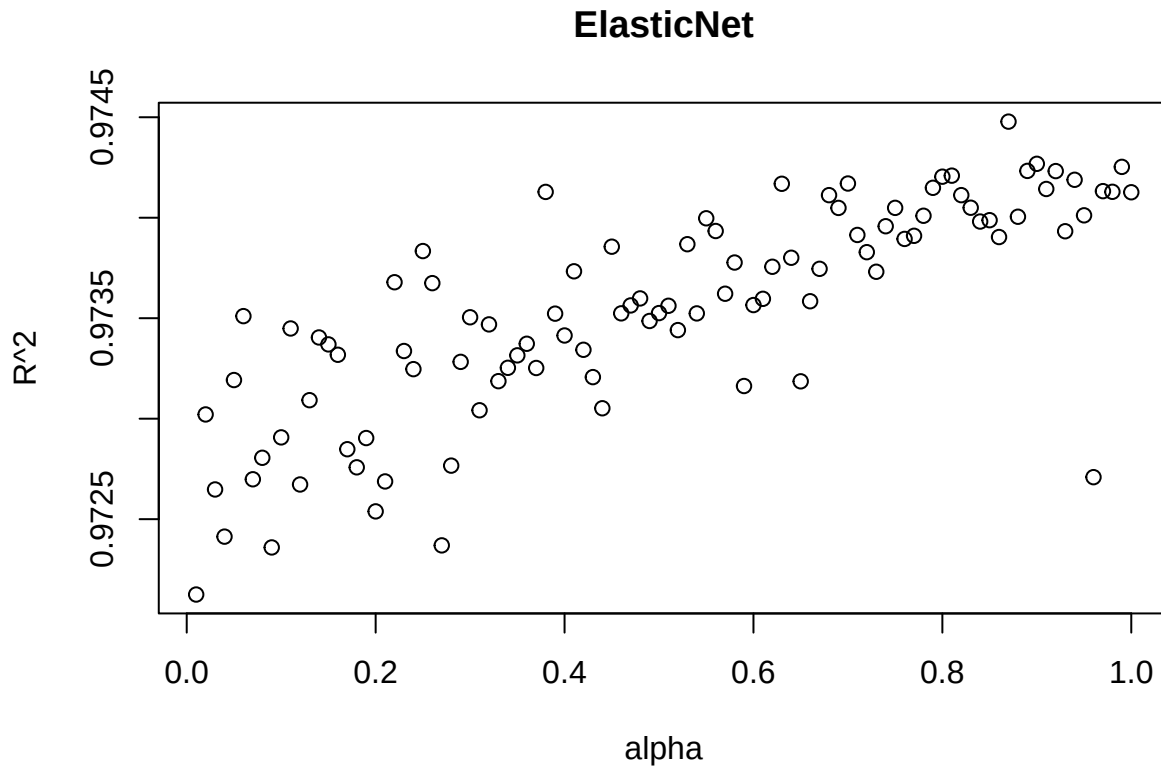
The MSE is 1.67.

e-)Perform Elastic Net and comment on the model performance

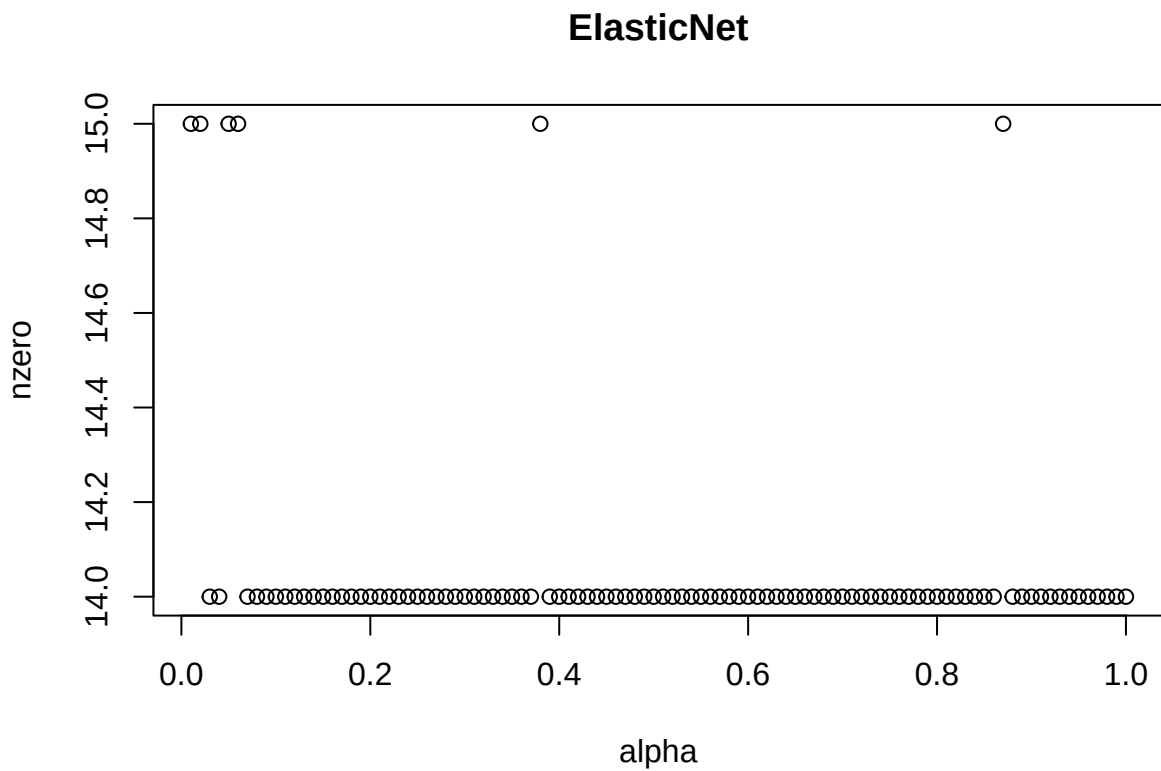
```
alpha_ivl = 0.01
fat_en_res = data.frame()
for (i in 1:floor(1/alpha_ivl)) {
  alpha = i*alpha_ivl
  fat_en = cv.glmnet(fat_training_dm, fat_training_response, alpha=alpha, nlambda=100, lambda.min.ratio=0.1)
  fat_en_lambda_min = fat_en$lambda.min
  fat_en_nzero = fat_en$nzero[fat_en$lambda == fat_en_lambda_min]
  fat_en_coefficients = predict(fat_en, s=fat_en_lambda_min, type="coefficients")
  fat_en_predicted = as.matrix(data.frame(1, fat_training_dm)) %*% fat_en_coefficients
  fat_training_sst = sum((fat_training_response - mean(fat_training_response))^2)
  fat_en_sse = sum((fat_training_response - fat_en_predicted)^2)
  fat_en_r_squared = 1 - (fat_en_sse/fat_training_sst)
  fat_en_res = rbind(
    fat_en_res,
    data.frame(alpha=alpha, nzero=fat_en_nzero, r_squared=fat_en_r_squared)
  )
}
```

```
}
```

```
plot(fat_en_res$alpha, fat_en_res$r_squared, main="ElasticNet", xlab="alpha", ylab="R^2")
```



```
plot(fat_en_res$alpha, fat_en_res$nzero, main="ElasticNet", xlab="alpha", ylab="nzero")
```

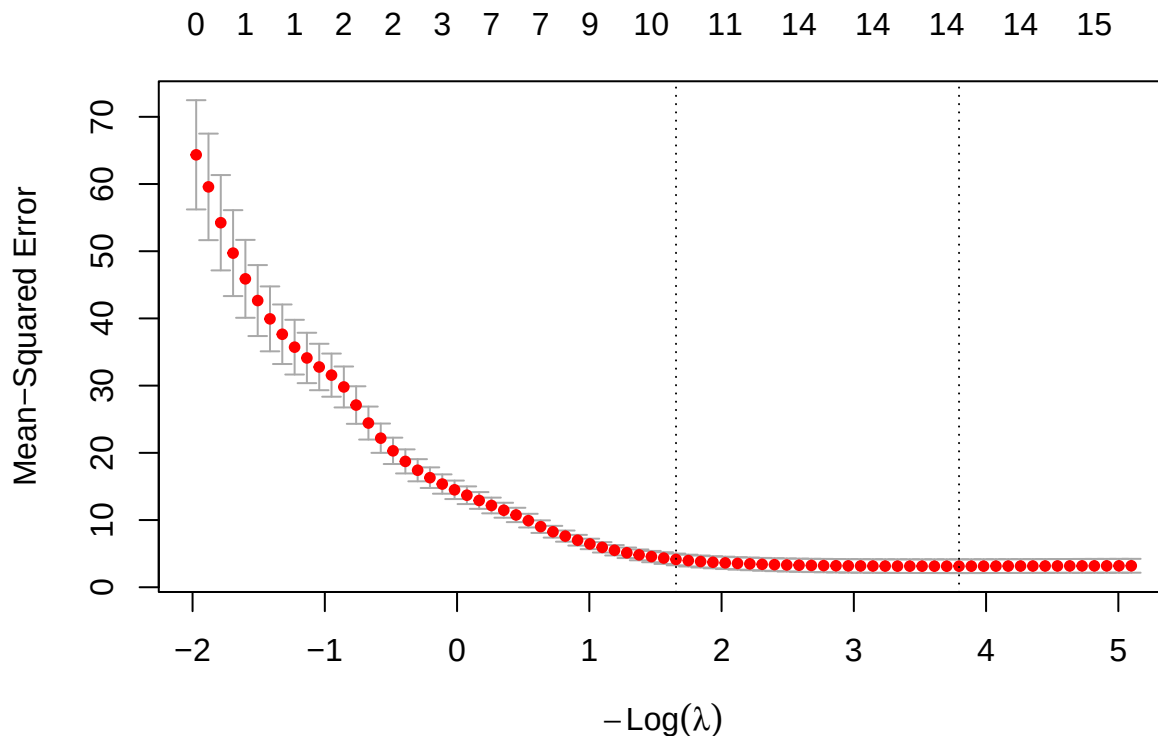


```
best_r_squared = max(fat_en_res$r_squared)
best_alpha = fat_en_res$alpha[fat_en_res$r_squared==best_r_squared]
fat_en = cv.glmnet(fat_training_dm, fat_training_response, alpha=best_alpha, nlambda=100, lambda.min.ratio=0.1)
fat_en_lambda_min = fat_en$lambda.min

print(fat_en)
```

```
##
## Call: cv.glmnet(x = fat_training_dm, y = fat_training_response, alpha = best_alpha, nlambda = 100)
##
## Measure: Mean-Squared Error
##
##      Lambda Index Measure      SE Nonzero
## min 0.02247   63   3.132 1.0238        14
## 1se 0.19097   40   4.136 0.8701        10
```

```
plot(fat_en,)
```



```
defaultSummary(data.frame(
  obs=fat_training_response,
  pred=c(predict(fat_en, s=fat_en_lambda_min, fat_training_dm))
))
```

```
##      RMSE Rsquared      MAE
## 1.2879647 0.9742276 0.9407137
```

The Elastic Net regression converges to the lasso.

The MSE is 1.66.

Use the test data sets to answer questions below

f-) Test the model's performances for part (a) to part (e) above and comment on model stability and performance. Which model would you choose and why?

```
fat_testing_dm = model.matrix(siri ~ ., fat_testing)[-c(1)]

# Linear Model
defaultSummary(data.frame(obs=fat_testing$siri, pred=predict(fat_lm, fat_testing)))

##          RMSE  Rsquared          MAE
## 4.2283189 0.8039551 1.6409942

# Stepwise Two Way Adjusted R-Squared
defaultSummary(data.frame(obs=fat_testing$siri, pred=predict(fat_ols_step_both$model, fat_testing)))

##          RMSE  Rsquared          MAE
## 4.3993612 0.7905642 1.7124544

# Ridge
defaultSummary(data.frame(obs=fat_testing$siri, pred=c(predict(fat_ridge, s=fat_ridge_lambda_min, fat_testing))),

##          RMSE  Rsquared          MAE
## 3.1810147 0.8813378 1.4960811

#Lasso
defaultSummary(data.frame(obs=fat_testing$siri, pred=c(predict(fat_lasso, s=fat_lasso_lambda_min, fat_testing))),

##          RMSE  Rsquared          MAE
## 3.1115598 0.8859824 1.4861887

# ElasticNet
defaultSummary(data.frame(obs=fat_testing$siri, pred=c(predict(fat_en, s=fat_en_lambda_min, fat_testing))),

##          RMSE  Rsquared          MAE
## 3.5984142 0.8507471 1.5411134
```

The R-squared are lower in test, which may suggest overfitting. The Lasso has the lowest RMSE, and MAE, and highest R-squared. I would use Lasso.

Problem 2

Use the fat data set in previous example by using the percentage of body fat, siri, as the response and the other variables, except brozek and density as potential predictors.

a-)Fit the same model in question 1-a but now using Huber's robust method. Comment on any substantial differences between this model and the least squares fit.

```
fat_huber = rlm(siri ~ ., fat_training)
fat_hampel = rlm(siri ~ ., fat_training, psi=psi.hampel)
fat_bisquare = rlm(siri ~ ., fat_training, psi=psi.bisquare)
fat_rlm = data.frame(
  idx = c(1:nrow(fat_training)),
  huber = fat_huber$w,
  hampel = fat_hampel$w,
  bisquare = fat_bisquare$w
)
fat_rlm = fat_rlm[order(fat_rlm$huber),]
head(fat_rlm)

##      idx      huber      hampel      bisquare
## 39    28 0.08343537 0.0000000 0.000000000
```

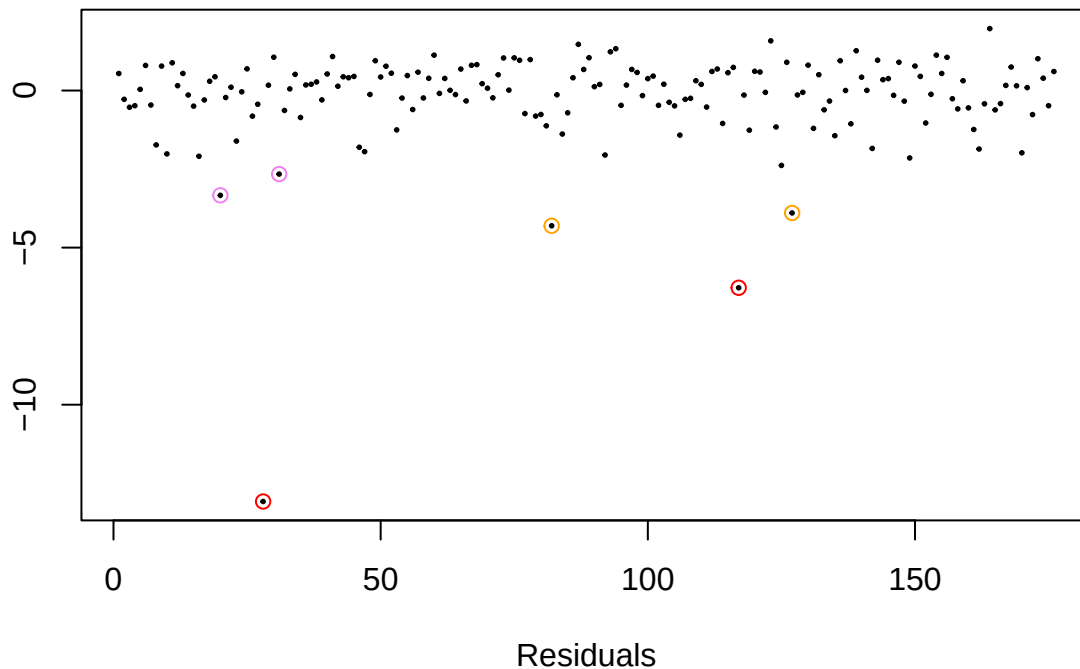
```
## 169 117 0.17382307 0.0000000 0.000000000
## 182 82 0.25350491 0.3101977 0.000000000
## 172 127 0.28007119 0.4166610 0.000000000
## 55 20 0.32722083 0.5168407 0.003551959
## 54 31 0.41032967 0.8400872 0.551215273
```

b-)Identify which two cases have the lowest weights in the Huber fit. What is unusual about these two points?

```
fat_outlier_idx = fat_rylm[1:6, "idx"]
fat_outlier_residuals = fat_huber$residuals[fat_outlier_idx]
```

```
plot(
  fat_huber$residuals,
  pch = 19,
  cex=0.25,
  xlab="Residuals",
  ylab = "",
)

points(fat_outlier_idx[1:2], fat_outlier_residuals[1:2], col="red")
points(fat_outlier_idx[3:4], fat_outlier_residuals[3:4], col="orange")
points(fat_outlier_idx[5:6], fat_outlier_residuals[5:6], col="violet")
```

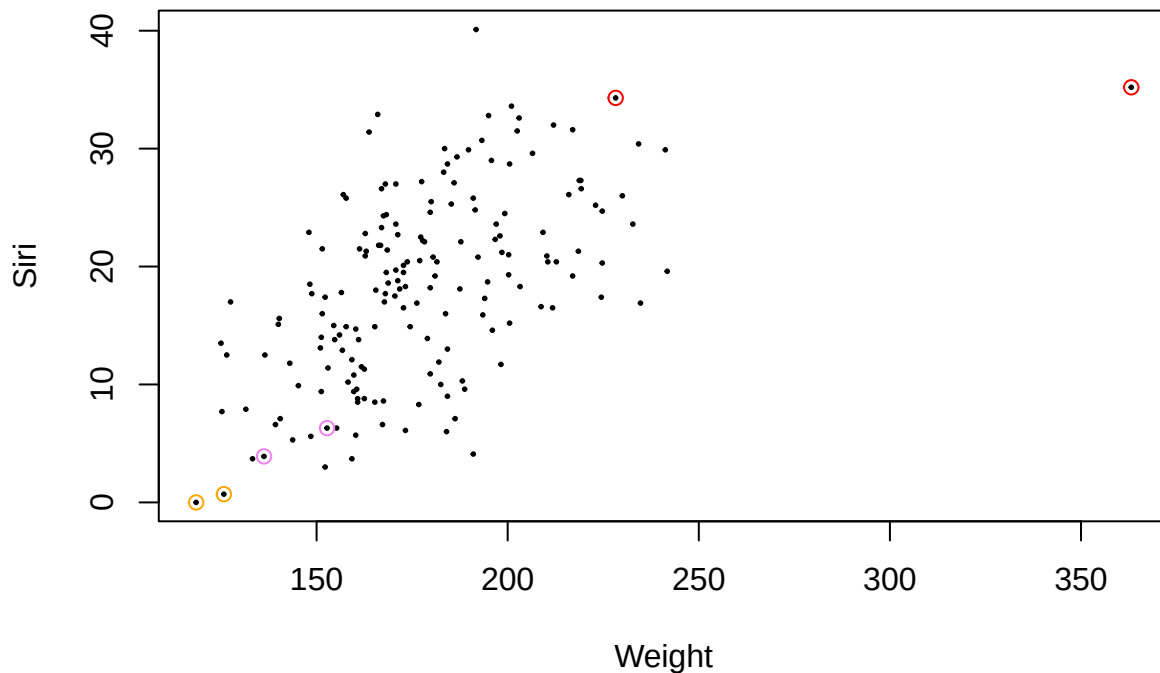


These points have the most extreme residual values.

```
plot(
  fat_training$weight,
  fat_training$siri,
  pch = 19,
  cex=0.25,
  main="Weight vs. Siri",
  xlab="Weight",
  ylab="Siri"
)
```

```
)
points(fat_training[fat_outlier_idx[1:2], "weight"], fat_training[fat_outlier_idx[1:2], "siri"], col="red",
points(fat_training[fat_outlier_idx[3:4], "weight"], fat_training[fat_outlier_idx[3:4], "siri"], col="orange",
points(fat_training[fat_outlier_idx[5:6], "weight"], fat_training[fat_outlier_idx[5:6], "siri"], col="violet",
```

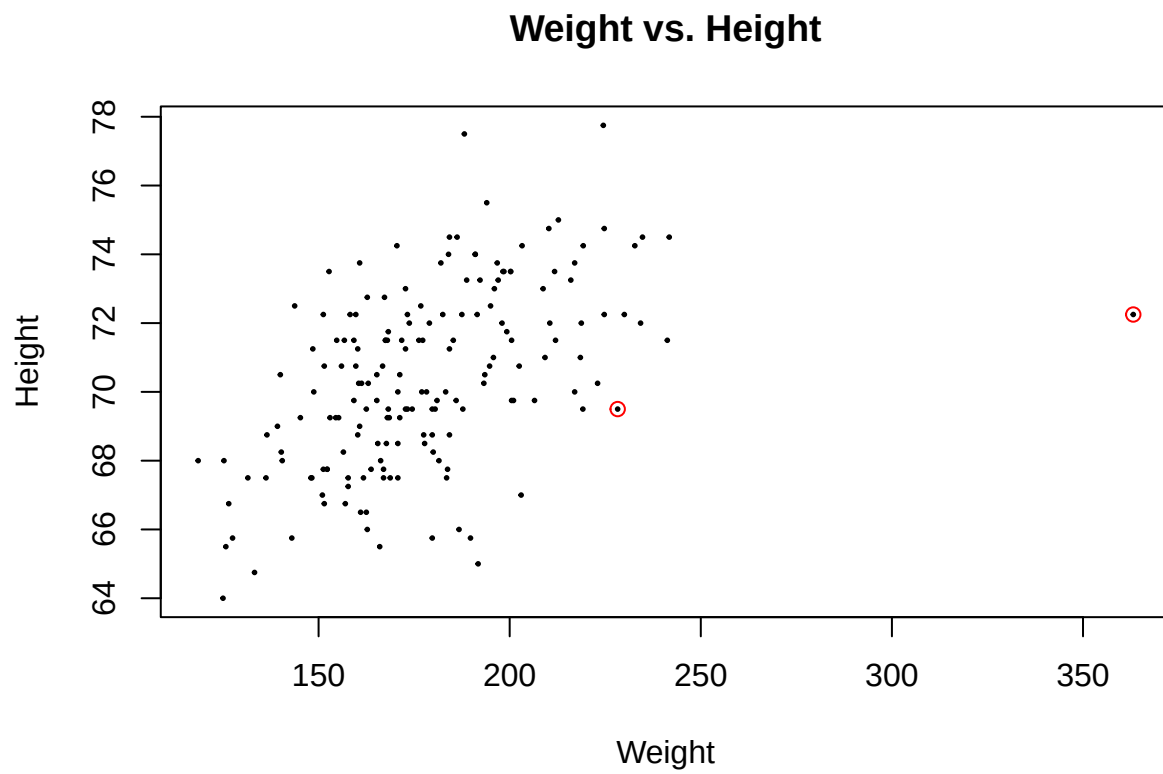
Weight vs. Siri



The most extreme huber weight corresponds to a row of maximum weight over all of the data.

c-) Plot weight (of the man) against height. Identify the two outlying cases. Are these the same as those identified in the previous question? Discuss.

```
plot(
  fat_training$weight,
  fat_training$height,
  pch = 19,
  cex=0.25,
  main="Weight vs. Height",
  xlab="Weight",
  ylab="Height"
)
points(
  fat_training[fat_outlier_idx[1:2], "weight"],
  fat_training[fat_outlier_idx[1:2], "height"],
  col="red"
)
```



The indices of the outliers point to at least one high leverage point in the weight vs height chart.